Unit 2: Perception and Motor Actions in ACT-R

2.1 ACT-R Interacting with the World

This unit will introduce some of the mechanisms which allow ACT-R to interact with the world, which for the purposes of the tutorial will be experiments presented via the computer. This is made possible with the addition of perceptual and motor modules which were originally developed by Mike Byrne as a separate system called ACT-R/PM but which are now an integrated part of the ACT-R architecture. These additional modules provide a model with visual, motor, auditory, and vocal capabilities based on human performance, and also include mechanisms for interfacing those modules to the world. The auditory, motor, and vocal modules are based upon the corresponding components of the EPIC architecture developed by David Kieras and David Meyer, but the vision module is based on visual attention work which was done in ACT-R. The interface to the world which we will use in the tutorial allows the model to interact with the computer i.e. process visual items presented, press keys, and move and click the mouse, for tasks which are created using tools built into ACT-R for generating user interfaces. It is also possible for one to extend that interface or implement new interfaces to allow models to interact with other environments, but that is beyond the scope of the tutorial.

2.1.1 Experiments

From this point on in the tutorial most of the models will be performing an experiment or task which requires interacting with the world in some way. That means that now instead of just running the model itself one will have to run the corresponding task for the model to perform instead, and that task will handle the running of the model. All of the tasks for the tutorial models have been written in both ANSI Common Lisp and Python 3. The tutorial will show how to run both versions of all the tasks, and the experiment description document in each unit will provide additional details on how those tasks are implemented. From the model's perspective, it does not matter whether the Lisp or Python version of the task is used, and the single model file included with the tutorial will run with either version producing the same results.

2.2 The First Experiment

The first experiment is very simple and consists of a display in which a single letter is presented. The participant's task is to press the key corresponding to that letter. When any key is pressed, the display is cleared and the experiment ends. This experiment can be run for either a human participant or for an ACT-R model, and the model in the demo2-model.lisp file in the tutorial is able to perform this task. If you wish to run the task for a human participant then you must have an ACT-R experiment window viewer running to see and interact with the task, and the ACT-R Environment includes such a viewer which will be available as long as you do not close the Control Panel window.[1]

[1] There is also an additional application included with the ACT-R software which will allow the experiment windows to be used through a browser. The readme file with the ACT-R standalone software contains instructions on how to run that if you do not want to use the ACT-R Environment application.

2.2.1 Running the Lisp version

The first thing you will need to do to run the Lisp version of the experiment is load the experiment code. There are two ways that can be done:

- You can use the "Load ACT-R code" button in the Environment just as you loaded the model files in unit 1. The file is called demo2.lisp and is found in the lisp directory of the ACT-R tutorial.

- You can call the actr-load[2] function directly from the ACT-R prompt specifying the location of that file. If the tutorial directory is still located with the rest of the ACT-R files from the standalone distribution then that would look like this:

```
? (actr-load "ACT-R:tutorial;lisp;demo2.lisp")
```

When you load that file it will also load the model which can perform the task from the demo2-model.lisp file in the tutorial/unit2 directory, and if you look at the top of the Control Panel you will see that it says DEMO2 under "Current Model". To run the experiment you will call the demo2-experiment function, and it has one optional parameter which indicates whether a human will be performing the task. As a first run you should perform the task yourself, and to do that you will evaluate the **demo2-experiment** function at the prompt in the ACT-R window and pass it a value of t (the Lisp symbol representing true):

```
? (demo2-experiment t)
```

When you enter that a window titled "Letter recognition" will appear with a letter in it (the window may be obscured by other open windows so you may have to arrange things to ensure you can see everything you want). When you press a key while that experiment window is the active window the experiment window will clear and that is the end of the experiment. The letter you typed will be returned by the **demo2-experiment** function.

2.2.2 Running the Python version

To run the Python version you should first run an interactive Python session on your machine (instructions on how to do that are not part of this tutorial and you will need to consult the Python documentation for details). For the examples in this tutorial we will assume that the directory containing the ACT-R Python modules (the tutorial/python directory) is either the current directory for the Python session or that it has been added to the Python search path, and we will also assume that the Python prompt is the three character sequence ">>>". Once your Python session is running there are two ways you can import the module:

- You can use import to directly import that module from the Python prompt:

```
>>> import demo2
```

[2]The Lisp load function could also be used, but that can vary based on the specific version of the software you are using and may require specifying the entire path instead of using the shortcut of "ACT-R:". The actr-load function works the same in all versions.

- If you first import the actr module instead, then you can use the “Import Python module” button in the Environment to pick a module to be imported into Python, and for this task the file is demo2.py in the python directory of the ACT-R tutorial.

Importing the demo2 module will also have ACT-R load the model for this task, which is found in the demo2-model.lisp file in the tutorial/unit2 directory, and if you look at the top of the Control Panel you will see that it now says DEMO2 under “Current Model”. To run the experiment you will call the **experiment** function in the demo2 module, and it has one optional parameter which indicates whether a human will be performing the task. As a first run you should perform the task yourself, and to do that you will call the **experiment** function at the Python prompt and pass it the value True:

```
>>> demo2.experiment(True)
```

When you enter that a window titled “Letter recognition” will appear with a letter in it (the window may be obscured by other open windows so you may have to arrange things to ensure you can see everything you want). When you press a key while that experiment window is the active window the experiment window will clear and that is the end of the experiment. The letter you typed will be returned by the **experiment** function.

2.2.3 Running the model in the experiment

You can run the model through the experiment by calling the function without including the true value that indicates a human participant. That would look like this for the two different versions:

```
? (demo2-experiment)
```

or

```
>>> demo2.experiment()
```

Regardless of which version you run, you will see the following trace of the model performing the task:

```
0.000   GOAL                SET-BUFFER-CHUNK GOAL (ISA READ-LETTERS STEP START) NIL
0.000   VISION              proc-display
0.000   VISION              SET-BUFFER-CHUNK VISUAL-LOCATION VISUAL-LOCATION0 NIL
0.000   VISION              visicon-update
0.000   PROCEDURAL          CONFLICT-RESOLUTION
0.000   PROCEDURAL          PRODUCTION-SELECTED FIND-UNATTENDED-LETTER
0.000   PROCEDURAL          BUFFER-READ-ACTION GOAL
0.050   PROCEDURAL          PRODUCTION-FIRED FIND-UNATTENDED-LETTER
0.050   PROCEDURAL          MOD-BUFFER-CHUNK GOAL
0.050   PROCEDURAL          MODULE-REQUEST VISUAL-LOCATION
0.050   PROCEDURAL          CLEAR-BUFFER VISUAL-LOCATION
0.050   VISION              Find-location
0.050   VISION              SET-BUFFER-CHUNK VISUAL-LOCATION VISUAL-LOCATION0
0.050   PROCEDURAL          CONFLICT-RESOLUTION
0.050   PROCEDURAL          PRODUCTION-SELECTED ATTEND-LETTER
0.050   PROCEDURAL          BUFFER-READ-ACTION GOAL
0.050   PROCEDURAL          BUFFER-READ-ACTION VISUAL-LOCATION
0.050   PROCEDURAL          QUERY-BUFFER-ACTION VISUAL
0.100   PROCEDURAL          PRODUCTION-FIRED ATTEND-LETTER
```

```
0.100   PROCEDURAL              MOD-BUFFER-CHUNK GOAL
0.100   PROCEDURAL              MODULE-REQUEST VISUAL
0.100   PROCEDURAL              CLEAR-BUFFER VISUAL-LOCATION
0.100   PROCEDURAL              CLEAR-BUFFER VISUAL
0.100   VISION                  Move-attention VISUAL-LOCATION0-1 NIL
0.100   PROCEDURAL              CONFLICT-RESOLUTION
0.185   VISION                  Encoding-complete VISUAL-LOCATION0-1 NIL
0.185   VISION                  SET-BUFFER-CHUNK VISUAL TEXT0
0.185   PROCEDURAL              CONFLICT-RESOLUTION
0.185   PROCEDURAL              PRODUCTION-SELECTED ENCODE-LETTER
0.185   PROCEDURAL              BUFFER-READ-ACTION GOAL
0.185   PROCEDURAL              BUFFER-READ-ACTION VISUAL
0.185   PROCEDURAL              QUERY-BUFFER-ACTION IMAGINAL
0.235   PROCEDURAL              PRODUCTION-FIRED ENCODE-LETTER
0.235   PROCEDURAL              MOD-BUFFER-CHUNK GOAL
0.235   PROCEDURAL              MODULE-REQUEST IMAGINAL
0.235   PROCEDURAL              CLEAR-BUFFER VISUAL
0.235   PROCEDURAL              CLEAR-BUFFER IMAGINAL
0.235   PROCEDURAL              CONFLICT-RESOLUTION
0.435   IMAGINAL                SET-BUFFER-CHUNK IMAGINAL (LETTER "V")
0.435   PROCEDURAL              CONFLICT-RESOLUTION
0.435   PROCEDURAL              PRODUCTION-SELECTED RESPOND
0.435   PROCEDURAL              BUFFER-READ-ACTION GOAL
0.435   PROCEDURAL              BUFFER-READ-ACTION IMAGINAL
0.435   PROCEDURAL              QUERY-BUFFER-ACTION MANUAL
0.485   PROCEDURAL              PRODUCTION-FIRED RESPOND
0.485   PROCEDURAL              MOD-BUFFER-CHUNK GOAL
0.485   PROCEDURAL              MODULE-REQUEST MANUAL
0.485   PROCEDURAL              CLEAR-BUFFER IMAGINAL
0.485   PROCEDURAL              CLEAR-BUFFER MANUAL
0.485   MOTOR                   PRESS-KEY KEY V
0.485   PROCEDURAL              CONFLICT-RESOLUTION
0.735   MOTOR                   PREPARATION-COMPLETE 0.485
0.735   PROCEDURAL              CONFLICT-RESOLUTION
0.785   MOTOR                   INITIATION-COMPLETE 0.485
0.785   PROCEDURAL              CONFLICT-RESOLUTION
0.885   KEYBOARD                output-key DEMO2 v
0.885   VISION                  proc-display
0.885   VISION                  visicon-update
0.885   PROCEDURAL              CONFLICT-RESOLUTION
0.970   VISION                  Encoding-complete VISUAL-LOCATION0-1 NIL
0.970   VISION                  No visual-object found
0.970   PROCEDURAL              CONFLICT-RESOLUTION
1.035   MOTOR                   FINISH-MOVEMENT 0.485
1.035   PROCEDURAL              CONFLICT-RESOLUTION
1.035   ------                  Stopped because no events left to process
```

Unlike the previous unit where we had to run the model using the Run button on the Control Panel, the code which implements this experiment automatically runs the model to do the task. It is not necessary that it operate that way, but it is often convenient to build the experiments for the models to do so, particularly in tasks like those used in later units where we will be running the models through the experiments many times to collect performance measures.

Looking at that trace we see production firing being intermixed with actions of the vision, imaginal, and motor modules as the model encodes the stimulus and issues a response. If you watch the window while the model is performing the task you will also see a red circle drawn. That is a debugging aid which

indicates the model's current point of visual attention, and can be turned off if you do not want to see it. You may also notice that the task always presents the letter "V". That is done so that it always generates the same trace. In the following sections we will look at how the model perceives the letter being presented, how it issues a response, and then briefly discuss some parameters in ACT-R that control things like the attention marker and the pseudo-random number generator.

2.3 Control and Representation

Before looking at the details of the new modules used in this unit we will first look at a difference in how the information for the task is represented compared to the unit 1 models. If you open the demo2-model.lisp file and look at the model definition you will find two chunk-types created for this model:

```
(chunk-type read-letters step)
(chunk-type array letter)
```

The chunk-type read-letters specifies one slot which is called step and will be used to track the current task state for the model. The other chunk-type, array, also has only one slot, which is called letter, and will hold a representation of the letter which is seen by the model.

In unit 1, the chunk in the **goal** buffer had slots which held all of the information relevant to performing the task. That approach is how ACT-R models were typically built in older versions of the architecture, but now a more distributed representation of the model's task information across two buffers is the recommended approach to modeling with ACT-R. The **goal** buffer should be used to hold control information – the internal representation of what the model is doing and where it is in the task. A different buffer, the **imaginal** buffer, should be used to hold a chunk which contains the information needed to perform the task. In the demo2 model, the **goal** buffer will hold a chunk based on the read-letters chunk-type, and the **imaginal** buffer will hold a chunk based on the array chunk-type.

2.3.1 The Step Slot

In this model, the step slot of the chunk in the **goal** buffer will maintain information about what the model is doing, and it is used to explicitly indicate which productions are appropriate at any time. This is often done when writing ACT-R models because it provides an easy means of specifying an ordering of the productions and it can make it easier to understand the way the model operates by looking at the productions. It is however not always necessary to do so, and there are other means by which the same control flow can be accomplished. In fact, as we will see in a later unit there can be consequences to keeping extra information in a buffer. However, because it does make the production sequencing in a model clearer you will see a slot named step (or something similar) in many of the models in the tutorial. As an additional challenge for this unit, you should try to modify the demo2 model so that it works without needing to maintain an explicit step marker and thus not need to use the **goal** buffer at all.

2.4 The Imaginal Module

The first new module we will describe in this unit is the imaginal module. This module has a buffer called **imaginal** which is used to create new chunks. These chunks will be the model's internal

representation of information – its internal image (hence the name). Like any buffer, the chunk in the **imaginal** buffer can be modified by the productions to build that representation using RHS modification actions as shown in unit 1.

An important issue with the **imaginal** buffer is how a chunk first gets into the buffer. Unlike the **goal** buffer's chunk which we have been specifying with the goal-focus command that places it there when the model starts running, the imaginal module will create the chunk for the **imaginal** buffer in response to a request from a production.

All requests to the imaginal module through the **imaginal** buffer are requests to create a new chunk. The imaginal module will create a new chunk using the slots and values provided in the request and place that chunk into the **imaginal** buffer. An example of this is shown in the action of the encode-letter production of the demo2 model:

```
(P encode-letter
   =goal>
      ISA         read-letters
      step        attend
   =visual>
      value       =letter
   ?imaginal>
      state       free
==>
   =goal>
      step        respond
   +imaginal>
      isa         array
      letter      =letter
)
```

We will come back to the condition of that production later. For now, we are interested in this request on the RHS:

```
   +imaginal>
      isa         array
      letter      =letter
```

This request to the **imaginal** buffer will result in the imaginal module creating a chunk which has a slot named letter that has the value of the variable =letter. We see the request and its results in these lines of the trace:

```
     0.235   PROCEDURAL             PRODUCTION-FIRED ENCODE-LETTER
...
     0.235   PROCEDURAL             MODULE-REQUEST IMAGINAL
...
     0.235   PROCEDURAL             CLEAR-BUFFER IMAGINAL
...
     0.435   IMAGINAL               SET-BUFFER-CHUNK IMAGINAL (LETTER "V")
```

When the encode-letter production fires it makes that request and automatically clears the buffer at that time as happens for all buffer requests. Then, we see that the imaginal module performs the action set-buffer-chunk showing the description of the chunk that is being created for the **imaginal** buffer, the same as we see with the goal module when it creates the chunk for the **goal** buffer when the goal-focus command is used.

Something to notice in the trace is that the buffer was not immediately set to have that chunk as a result of the request. It took .2 seconds before the chunk was made available in the buffer – the request happened at time 0.235 seconds but the set-buffer-chunk action did not happen until time 0.435 seconds. This is an important aspect of the imaginal module – it takes time to build a representation. The amount of time that it takes the imaginal module to create a chunk is a fixed cost, and the default time is .2 seconds (that can be changed with a parameter). In addition to the time cost, the imaginal module is only able to create one new chunk at a time. That does not affect this model because it is only creating the one new chunk in the **imaginal** buffer, but in models which require a richer representation, that bottleneck may be a constraint on how fast the model can perform the task. In such situations, one should first verify that the module is available to create a new chunk before making a request. That is done with a query of the buffer on the LHS, and that is done in the encode-letter production seen above:

```
   ?imaginal>
      state       free
```

Additional information about querying modules will be described later in the unit.

In this model, the **imaginal** buffer will hold a chunk which contains a representation of the letter which the model reads from the screen. For this simple task, that representation is not necessary because the model could use the information directly from the **visual** buffer to do the task, but for most tasks there will be more than one piece of information which must be acquired incrementally which requires storing the intermediate values as it progresses. Thus, for demonstration purposes, this model records that information in the **imaginal** buffer's chunk.

2.5 The Vision Module

Many tasks involve interacting with visible stimuli and the vision module provides the model with a means for acquiring visual information. It is designed as a system for modeling visual attention that assumes there are lower-level perceptual processes that generate the representations with which it operates, but it does not model those perceptual processes in detail. The experiment generation tools in ACT-R create tasks which can provide representations of text, lines, and button features from the displays it creates to the vision module. The ability to provide visual feature information to the vision module is also available to modelers for creating new visual features, but that is beyond the scope of the tutorial.

The vision module has two buffers. There is a **visual** buffer that holds a chunk which represents an object in the visual scene and a **visual-location** buffer that holds a chunk which represents the location of an object in the visual scene. In the demo2 model, visual actions are performed in the productions **find-unattended-letter** and **attend-letter**.

2.5.1 Visual-Location buffer

The **find-unattended-letter** production applies whenever the **goal** buffer's chunk has the value start in the step slot (which is the value in the step slot of the chunk initially created for the **goal** buffer):

```
(P find-unattended-letter
   =goal>
      ISA         read-letters
      step        start
 ==>
   +visual-location>
      :attended    nil
   =goal>
      step        find-location
)
```

It makes a request of the **visual-location** buffer and it changes the **goal** buffer chunk's step slot to the value find-location. It is important to note that those values for the step slot of the **goal** buffer chunk are arbitrary. The values used in this model were chosen to help make clear what the model is doing, but the model would continue to operate the same if all of the corresponding references were consistently changed to other values instead.

A **visual-location** request asks the vision module to find the location of an object in its visual scene that meets the specified requirements, build a chunk to represent the location of that object if one exists, and place that chunk in the **visual-location** buffer.

The following portion of the trace reflects the actions performed by this production:

```
     0.050   PROCEDURAL            PRODUCTION-FIRED FIND-UNATTENDED-LETTER
     0.050   PROCEDURAL            MOD-BUFFER-CHUNK GOAL
     0.050   PROCEDURAL            MODULE-REQUEST VISUAL-LOCATION
     0.050   PROCEDURAL            CLEAR-BUFFER VISUAL-LOCATION
     0.050   VISION                Find-location
     0.050   VISION                SET-BUFFER-CHUNK VISUAL-LOCATION VISUAL-LOCATION0
```

We see the notice of the **visual-location** request and the automatic clearing of the **visual-location** buffer due to the request being made by the production. Then the vision module reports that it is finding a location, and after that it places a chunk into the buffer. Notice that there was no time involved in handling the request – all of those actions took place at time 0.050 seconds. The **visual-location** requests always finish immediately which reflects the assumption that there is a perceptual system within the vision module operating in parallel with the procedural module that can make these visual features immediately available.

If you run the model through the task again and step through the model's actions using the Stepper you can use the "Buffers" tool to see that the chunk **visual-location0-1** will be in the **visual-location** buffer after that last event:

```
VISUAL-LOCATION: VISUAL-LOCATION0-1 [VISUAL-LOCATION0]
VISUAL-LOCATION0-1
   KIND  TEXT
   VALUE  TEXT
   COLOR  BLACK
   HEIGHT  10
   WIDTH  7
   SCREEN-X  430
   SCREEN-Y  456
   DISTANCE  1080
   SIZE  0.19999999
```

There are a lot of slots in the chunk placed into the **visual-location** buffer, and when making the request to find a location any of those slots can be used to constrain the results. However, we will not need to do so for this unit and instead rely upon the ability of the vision module to remember visual features it has previously attended. Unit 3 will look in more detail at those slots and how to construct **visual-location** requests to search for items.

2.5.1.1 The attended request parameter

If we look at the request which was made of the **visual-location** buffer in the **find-unattended-letter** production:

```
   +visual-location>
      :attended    nil
```

we see that all it consists of is ":attended nil" in the request. This **:attended** specification is called a request parameter. It is similar to specifying a slot in the request, but it does not correspond to a slot in any chunk or chunk-type specification in the model. Request parameters always start with the ":" character which is what distinguishes it from a slot. They can be provided in a request to a buffer regardless of any chunk-type that is specified (or when no chunk-type is specified as is the case here), and they are used to supply additional information to the module about a request.

For a visual-location request one can use the :**attended** request parameter to specify whether the vision module should find the location of an object which the model has previously looked at (attended to) or not. If it is specified as **nil**, then the request is for a location which the model has not attended, and if it is specified as **t**, then the request is for a location which has been attended previously. There is also a third option, **new**, which means that the model has not attended to the location and that the object has also recently appeared in the visual scene.

2.5.2 The attend-letter production

The **attend-letter** production applies when the goal step is find-location, there is a chunk in the **visual-location** buffer, and the vision module is not currently active with respect to the **visual** buffer:

```
(P attend-letter
   =goal>
      ISA         read-letters
      step        find-location
   =visual-location>
   ?visual>
```

```
      state       free
==>
   +visual>
      cmd         move-attention
      screen-pos  =visual-location
   =goal>
      step        attend
)
```

On the LHS of this production are two tests that have not been used in previous models. The first of those is a test of the **visual-location** buffer which has no constraints specified for the slots of the chunk in that buffer. All that is necessary for this production is that there is a chunk in the buffer – the details of its slots and values do not matter. The other is a query of the **visual** buffer.

2.5.3 Testing a module's state

On the LHS of **attend-letter** a query is made of the **visual** buffer to test that the **state** of the vision module is **free**. All buffers will respond to a query for their module's **state** and the possible values for that query are **busy** or **free**. The test of **state free** is a check to make sure the buffer being queried is available for a new request through the indicated buffer. If the **state** is **free**, then it is safe to issue a new request through that buffer, but if it is **busy** then it is usually not safe to do so.

You can use the "Buffers" tool in the Control Panel to see the current status of a buffer's queries in addition to the chunk it contains. If you press the button labeled "Status" at the top right on the buffer viewer, that will change the information shown from the contents of the buffer to its status information. That shows the standard queries for each buffer along with the current value (either **t** or **nil**) for such a query at this time as well as any additional queries which may be specific to that buffer (details on all of the queries for each buffer can be found in the reference manual).

2.5.3.1 Jamming a module

Typically, a module is only able to handle one request to a buffer at a time, and that is the case for both the **imaginal** and **visual** buffers which require some time to produce a result. Since all of the model's modules operate in parallel it might be possible for the procedural module to select a production which makes a request to a module that is still working on a previous request. If a production were to fire at such a point and issue a request to a module which is currently busy and only able to handle one request at a time, that is referred to as "jamming" the module. When a module is jammed, it will output a warning message in the trace to let you know what has happened. What a module does when jammed varies from module to module. Some modules ignore the new request, whereas others abandon the previous request and start the new one. As a general practice it is best to avoid jamming modules.

Note that we did not query the state of the **visual-location** buffer in the **find-unattended-letter** production before issuing the **visual-location** request. That is because we know that those requests always complete immediately and thus the state of the vision module for the **visual-location** buffer is always free. We did however test the state of the imaginal module before making the request to the **imaginal** buffer in the **encode-letter** production. That query is not really necessary in this model because that is the only request to the imaginal module in the model and that production will not fire again

because of the change to the **goal** buffer chunk's step slot. Thus there is no risk of jamming the imaginal module in this model, but omitting queries which appear to be unnecessary can be a risky practice. It is always a good idea to query the state in every production that makes a request that could potentially jam a module even if you think that it will not happen because of the structure of the other productions in the model. Doing so makes it clear to anyone else who may read the model, and it also protects you from problems if you decide later to apply that model to a different task where the assumption which avoids the jamming no longer holds.

2.5.3.2 Strict Safety

In fact, like the strict harvesting mechanism described in unit 1 for automatically clearing buffers, there is also a "strict safety" mechanism which will automatically add a query for state free to a production that makes a request to a buffer which does not already have a query for that buffer (except for the **goal**, **retrieval**, and **visual-location** buffers which do not lead to jamming). That automatic query should prevent jamming in most cases, but explicitly adding queries to productions is still the recommended approach for readability of the model. The models in the tutorial will all include explicit queries and not rely upon the strict safety mechanism.

2.5.4 Chunk-type for the visual-location buffer

You may have noticed that we did not specify a chunk-type with either the request to the **visual-location** buffer in the **find-unattended-letter** production or its testing in the condition of the **attend-letter** production. That was because we didn't specify any slots in either of those places (recall that :attended is a request parameter and not a slot) thus there is no need to specify a chunk-type for verification that the slots used are correct. If we did need to request or test specific features with the **visual-location** buffer there is a chunk-type named visual-location which one can use to do so which has the slots shown above for the chunk in the **visual-location** buffer.

2.5.5 Visual buffer

On the RHS of the **attend-letter** production it makes a request of the **visual** buffer which specifies two slots: cmd and screen-pos:

```
+visual>
   cmd         move-attention
   screen-pos  =visual-location
```

Unlike the other buffers which we have seen so far, the **visual** buffer is capable of performing different actions in response to a request. The cmd slot in a request to the **visual** buffer indicates which specific action is being requested. In this request that is the value move-attention which indicates that the production is asking the vision module to move its attention to some location, create a chunk which encodes the object that is there, and place that chunk into the **visual** buffer. The location to which the module should move its attention is specified in the screen-pos (short for screen-position) slot of the request. In this production, that location is the chunk that is in the **visual-location** buffer (remember that

the condition specifying a buffer test also binds the variable naming the buffer to the chunk it contains). The following portion of the trace shows this request and the results:

```
     0.100   PROCEDURAL             PRODUCTION-FIRED ATTEND-LETTER
...
     0.100   PROCEDURAL             MODULE-REQUEST VISUAL
...
     0.100   PROCEDURAL             CLEAR-BUFFER VISUAL
     0.100   VISION                 Move-attention VISUAL-LOCATION0-1 NIL
...
     0.185   VISION                 Encoding-complete VISUAL-LOCATION0-1 NIL
     0.185   VISION                 SET-BUFFER-CHUNK VISUAL TEXT0
```

The request to move-attention is made at time 0.100 seconds and the vision module reports receiving that request at that time as well. Then 0.085 seconds pass before the vision module reports that it has completed encoding the object at that location, and it places a chunk into the **visual** buffer at time 0.185 seconds. Those 85 ms represent the time to shift attention and create the chunk representing the visual object. Altogether, counting the two production firings (one to request the location and one to harvest the result and request the attention shift) and the 85 ms to execute the attention shift and object encoding, it takes 185 ms to create the chunk that encodes the letter on the screen.

If you step through the model you will find this chunk in the **visual** buffer after those actions have occurred:

```
VISUAL: VISUAL-CHUNK0 [TEXT0]
VISUAL-CHUNK0
   SCREEN-POS  VISUAL-LOCATION0-0
   VALUE  "V"
   COLOR  BLACK
   HEIGHT  10
   WIDTH  7
   TEXT  T
```

Most of the chunks created for the **visual** buffer by the vision module are going to have a common set of slots. Those will include the **screen-pos** slot which holds the location chunk which represents where the object is located (which will typically have the same information as the location to which attention was moved) and then **color**, **height**, and **width** slots which hold information about the visual features of the object that was attended. In addition, depending on the details of the object which was attended, other slots may provide more details. When the object is text from an experiment window, like this one, the **value** slot will hold a string that contains the text encoded from the screen. We will describe some of the chunk-types that specify the slots for visual items below.

After a chunk has been placed in the **visual** buffer the model harvests that chunk with the **encode-letter** production:

```
(P encode-letter
   =goal>
      ISA         read-letters
      step        attend
   =visual>
      value       =letter
```

```
   ?imaginal>
      state       free
==>
   =goal>
      step        respond
   +imaginal>
      isa         array
      letter      =letter
)
```

It makes a request to the **imaginal** buffer to create a new chunk which will hold a representation of the letter as was described in the section above on the imaginal module.

2.5.6 Chunk-types for the visual buffer

As with the **visual-location** buffer you may have noticed that we also didn't specify a chunk-type with the request of the **visual** buffer in the **attend-letter** production or the harvesting of the chunk in the **encode-letter** production. Unlike the **visual-location** buffer, there actually are slots specified in both of those cases for the **visual** buffer. Thus, for added safety we should have specified a chunk-type in both of those places to validate the slots being used.

For the chunks placed into the **visual** buffer by the vision module there is a chunk-type called visual-object which specifies the slots: screen-pos, value, status, color, height, and width. For the objects which the vision module encodes from the experiment window interface the visual-object chunk-type is always an acceptable choice. Therefore a safer specification of the **encode-letter** production would include the chunk-type declaration shown here in red:

```
(P encode-letter
   =goal>
      ISA         read-letters
      step        attend
   =visual>
      ISA         visual-object
      value       =letter
   ?imaginal>
      state       free
==>
...
)
```

For the request to the **visual** buffer there are a couple of options available for how to include a chunk-type declaration to verify the slots. The first option is to use the chunk-type named vision-command which includes all of the slots for all of the requests which can be made to the **visual** buffer:

```
   +visual>
      isa         vision-command
      cmd         move-attention
      screen-pos  =visual-location
```

Because that contains slots for all of the different requests which the **visual** buffer can handle it is not as safe as a more specific chunk-type which only contains the slots for the specific command being used. For the move-attention command there is another chunk-type called move-attention which only contains the slots that are valid for the move-attention request. Therefore, a more specific declaration for the request to the **visual** buffer for the vision module to move attention to an object would be:

```
   +visual>
      isa         move-attention
      cmd         move-attention
      screen-pos  =visual-location
```

Specifying move-attention twice in that request looks a little awkward which is why it wasn't included in the original production. Later in the tutorial we will describe a way to avoid that redundancy and still maintain the safety of the chunk-type declaration.

2.5.7 Other Vision module actions

If you look closely at the trace you will find that there are seven other events which are attributed to the vision module that we have not yet described:

```
     0.000   VISION                PROC-DISPLAY
     0.000   VISION                SET-BUFFER-CHUNK VISUAL-LOCATION VISUAL-LOCATION0 NIL
     0.000   VISION                visicon-update
...
     0.885   VISION                PROC-DISPLAY
     0.885   VISION                visicon-update
...
     0.970   VISION                Encoding-complete VISUAL-LOCATION0-1 NIL
     0.970   VISION                No visual-object found
```

These actions represent activity which the vision module has performed without being requested to do so. The ones which indicate actions of proc-display and visicon-update are notices of internal updates which may be useful for the modeler to know, but are not directly relevant to the model itself. The proc-display actions indicate that something has happened which has caused the vision module to reprocess the information which is available to it. The one at time 0 happens because that is when the model first begins to interact with the display, and the one at time 0.885 seconds occurs because when the model pressed the key the screen was erased. Those are followed at the same times (though not necessarily immediately) by actions which say visicon-update. The visicon-update action is an indication that the reprocessing of the visual information resulted in an actual change to the information available in the vision module.

The other event at time 0 is the result of a mechanism in the vision module which we will not discuss until the next tutorial unit, so for now you should just ignore it. The actions at time 0.970 seconds will be described in the next section.

2.5.8 Visual Re-encoding

The two lines in the trace of the model at time .970 performed by the vision module were not the result of a request in a production:

```
0.970   VISION                Encoding-complete VISUAL-LOCATION0-1 NIL
0.970   VISION                No visual-object found
```

The first of those is an encoding-complete action as we saw above when the module was requested to move-attention to a location. This encoding-complete is triggered by the proc-display which occurred at time 0.885 seconds in response to the screen being cleared after the key press. If the vision module is attending to a location when it reprocesses the visual scene it will automatically re-encode the information at the attended location to encode any changes that may have occurred there. This re-encoding takes 85 ms just as an explicit request to attend an item does. If the visual-object chunk representing that item is still in the **visual** buffer it will be updated to reflect any changes. If there is no longer a visual item on the display at the location where the model is attending (as is the case here) then the trace will show a line indicating that no object was found. That will also result in the vision module noting a failure in the **visual** buffer.

While this automatic re-encoding process of the vision module is a useful component of the module, it does require that you be careful when writing models that process changing displays. In particular, since the module is **busy** while the re-encoding is occurring, the vision module cannot handle a new attention shift. This is one reason why it is important to query the visual **state** before all visual requests to avoid jamming the vision module – there may be activity other than that which is requested explicitly by the productions.

2.6 Learning New Chunks

This process of seeking the location of an object in one production, switching attention to the object in a second production, and harvesting the object in a third production is a common process in ACT-R models for handling perceptual information. Something to keep in mind about that processing is that this is one way in which ACT-R can acquire new declarative chunks. As was noted in the previous unit, the declarative memory module stores the chunks which are cleared from buffers, and that includes the perceptual buffers. Thus, as those perceptual chunks are cleared from their buffers, they will be recorded in the model's declarative memory.

2.7 The Motor Module

When we speak of motor actions in ACT-R we are only concerned with hand movements. It is possible to extend the motor module to other modes of action, but the provided motor module is built around controlling a pair of hands. In this unit the model will only be performing finger presses on a virtual keyboard, but there are also actions available for the model's hands to use a virtual mouse or joystick. It is also possible for the modeler to create additional motor capabilities and new devices to interact with, but that is beyond the scope of the tutorial.

The buffer for interacting with the motor module is called the **manual** buffer. Unlike other buffers however, the **manual** buffer will not have any chunks placed into it by its module. It is only used to issue

commands and to query the state of the motor module. As with the vision module, you should always check to make sure that the motor module is **free** before making any requests to avoid jamming it. The **manual** buffer query to test the state of the module works the same as the one described for the vision module:

```
?manual>
   state free
```

That query will be true when the module is available.

The motor module actually has a more complex set of internal states than just **free** or **busy** because there are multiple stages in performing the motor actions. By testing those internal states it is possible to make a new request to the motor module before the previous one has fully completed if it does not conflict with the ongoing action. We will not be describing the details of those internal states in the tutorial (they are described in the reference manual), and testing the overall state of the module will be sufficient for performing all of the **manual** buffer requests used in the tutorial models.

The **respond** production from the demo2 model shows the **manual** buffer in use:

```
(P respond
   =goal>
      ISA         read-letters
      step        respond
   =imaginal>
      isa         array
      letter      =letter
   ?manual>
      state       free
==>
   =goal>
      step        done
   +manual>
      cmd         press-key
      key         =letter
)
```

This production fires when the letter slot of the chunk in the **imaginal** buffer has a value, the **goal** buffer chunk's step slot has the value respond, and the **manual** buffer indicates that the motor module is free. A request is made to press the key corresponding to the letter from the letter slot of the chunk in the **imaginal** buffer and the step slot of the chunk in the **goal** buffer is changed to done.

Because there are many different actions which the motor module is able to perform, when making a request to the **manual** buffer a slot named cmd is used to indicate which action to perform. The **press-key** action used here assumes that the model's hands are located over the home row on the keyboard (which they are by default when using the provided experiment interface). From that position a press-key request will move the appropriate finger to touch type the character specified in the key slot of the request and then return that finger to the home row position.

Here are the events related to the **manual** buffer request from that production firing:

```
     0.485   PROCEDURAL            PRODUCTION-FIRED RESPOND
...
     0.485   PROCEDURAL            MODULE-REQUEST MANUAL
...
     0.485   PROCEDURAL            CLEAR-BUFFER MANUAL
     0.485   MOTOR                 PRESS-KEY KEY V
...
     0.735   MOTOR                 PREPARATION-COMPLETE 0.485
...
     0.785   MOTOR                 INITIATION-COMPLETE 0.485
...
     0.885   KEYBOARD              output-key DEMO2 v
...
     1.035   MOTOR                 FINISH-MOVEMENT 0.485
```

When the production is fired at time 0.485 seconds a request is made to press the key, the buffer is automatically cleared[3], and the motor module indicates that it has received a request to press the "v" key. However, it takes 250ms to prepare the features of the movement (preparation-complete), 50ms to initiate the action (initiation-complete), another 100ms until the key is actual struck and detected by the keyboard (output-key), and finally it takes another 150ms for the finger to return to the home row and be ready to move again (finish-movement). Thus the time of the key press is at 0.885 seconds, however the motor module is still busy until time 1.035 seconds. The numbers shown after the motor actions of preparation-complete, initiation-complete, and finish-movement are the time of the request to which they correspond for reference. The **press-key** request does not model the typing skills of an expert typist, but it does represent someone who is able to touch type individual letters competently without looking, at about 40 words per minute, which is usually a sufficient mechanism for modeling average performance in simple keyboard response tasks.

2.7.1 Motor module chunk-types

Like the **visual-location** and **visual** buffer requests the production which makes the request to the **manual** buffer did not specify a chunk-type. The **manual** buffer request is very similar to the request to the **visual** buffer. It has a slot named cmd which contains the action to perform and then additional slots as necessary to specify details for performing that action. The options for declaring a chunk-type in the request are also very similar to those for the **visual** buffer.

One option is to use the chunk-type named motor-command which includes all of the slots for all of the requests which can be made to the **manual** buffer:

```
+manual>
   isa         motor-command
   cmd         press-key
   key         =letter
```

Another is to use a more specific chunk-type named press-key that only has the valid slots for the press-key action (cmd and key):

```
+manual>
```

[3]Even though the motor module does not put chunks into its buffer, the procedural module still performs the clear action since it treats all buffers the same because it does not know the details of how other modules operate.

```
      isa         press-key
      cmd         press-key
      key         =letter
```

Again, that repetition is awkward and we will come back to that later in the tutorial.

2.8 Strict Harvesting

As mentioned in unit 1, the "strict harvesting" mechanism will implicitly clear a chunk from a buffer if the buffer is tested on the LHS of a production and that buffer is not modified on the RHS of the production. This mechanism is displayed in the events of the **attend-letter**, **encode-letter**, and **respond** productions which test and do not modify the **visual-location**, **visual**, and **imaginal** buffers respectively:

```
     0.100   PROCEDURAL            PRODUCTION-FIRED ATTEND-LETTER
...
     0.100   PROCEDURAL            CLEAR-BUFFER VISUAL-LOCATION
...
     0.235   PROCEDURAL            PRODUCTION-FIRED ENCODE-LETTER
...
     0.235   PROCEDURAL            CLEAR-BUFFER VISUAL
...
     0.485   PROCEDURAL            PRODUCTION-FIRED RESPOND
...
     0.485   PROCEDURAL            CLEAR-BUFFER IMAGINAL
```

If one wants to keep a chunk in a buffer after a production fires without modifying the chunk then it is valid to specify an empty modification to do so. For example, if one wanted to keep the chunk in the **visual** buffer after **encode-letter** fired we would only need to add an =visual> action to the RHS:

```
(P encode-letter-and-keep-chunk-in-visual-buffer
   =goal>
      ISA         read-letters
      step        attend
   =visual>
      value       =letter
   ?imaginal>
      state       free
==>
   =goal>
      step        respond
   +imaginal>
      isa         array
      letter      =letter
   =visual>
)
```

The strict harvesting mechanism also applies to the buffer failure query. A production which makes a query for buffer failure will also result in an automatic clearing of the buffer. That is done because clearing the buffer also clears the failure status, and having that automatically cleared makes it easier to write a model that handles failure conditions since the query will be false immediately after the production that tested it fires preventing it from firing again until there is another failure.

2.9 More ACT-R Parameters

The model code description document for unit 1 introduced the **sgp** command for setting ACT-R parameters. In the demo2 model the parameters are set like this:

```
(sgp :seed (123456 0))
(sgp :v t :show-focus t :trace-detail high)
```

All of these parameters are used to control how the software operates and do not affect the model's performance of the task. These settings are used to make this model a good example for showing the visual and motor actions. The :trace-detail parameter was described in the unit 1 code document and setting it to high ensures that all of the details are shown in the trace. The others are described in detail in the code document for this unit, but we will describe the :v (verbose) parameter briefly here since it is an important one. The :v parameter controls whether the trace of the model is displayed. If :v is **t** then the trace is displayed and if :v is set to **nil** the trace is not displayed. The software can run significantly faster when the trace is turned off, and that will be important in later units when we are running the models through the experiments multiple times to collect data.

2.10 Unit 2 Assignment

Your assignment is to write a model which uses the visual and motor capabilities introduced in the demo2 model to perform a slightly more complex experiment. The new experiment presents three letters on the screen, and two of those letters will be the same. The participant's task is to press the key that corresponds to the letter that is different from the other two. The code to perform the experiment is found in the unit2 file in the Lisp and Python directories. By default it will load the model found in the tutorial/unit2/unit2-assignment-model.lisp file. That initial model file only contains two chunk-type definitions, creates a chunk called start, and has a goal-foucs to specifies the initial chunk for the **goal** buffer.

The experiment is run very much like the demonstration experiment described earlier, and we will repeat the detailed instructions for how to load and run an experiment again here. In future units however we will assume that you are familiar with the process and only indicate the functions necessary to run the experiments.

2.10.1 Running the Lisp version

The first thing you will need to do to run the Lisp version of the experiment is load the experiment code. That can be done using the "Load ACT-R code" button in the Environment. The file is called unit2.lisp and is found in the tutorial/lisp directory of the ACT-R software. When you load that file it will also load the model from the unit2-assignment-model.lisp file in the tutorial/unit2 directory, and if you look at the top of the Control Panel after loading the file you will see that it says UNIT2 under "Current Model". To run the experiment you will call the unit2-experiment function, and it has one optional parameter which indicates whether a human participant will be performing the task. As a first run you should perform the task yourself, and to do that you will evaluate the **unit2-experiment** function at the prompt in the ACT-R window and pass it a value of t (the Lisp symbol representing true):

```
? (unit2-experiment t)
```

When you enter that, a window titled “Letter difference” will appear with three letters in it. When you press a key while that experiment window is the active window the experiment will record your key press and determine if it is the correct response. If the response is correct unit2-experiment will return t:

```
? (unit2-experiment t)
T
?
```

and if it is incorrect it will return nil:

```
? (unit2-experiment t)
NIL
?
```

To run the model through the task you call the unit2-experiment function without providing a parameter. With the initial model that will result in this which returns nil indicating an incorrect response (because the model did not make a response):

```
? (unit2-experiment)
     0.000   GOAL                 SET-BUFFER-CHUNK GOAL (ISA READ-LETTERS STEP START) NIL
     0.000   VISION               SET-BUFFER-CHUNK VISUAL-LOCATION VISUAL-LOCATION0 NIL
     0.000   VISION               visicon-update
     0.000   PROCEDURAL           CONFLICT-RESOLUTION
     0.500   PROCEDURAL           CONFLICT-RESOLUTION
     0.500   ------               Stopped because no events left to process
NIL
```

2.10.2 Running the Python version

To run the Python version you should first run an interactive Python session on your machine (you can use the same session used for the demonstration experiment if it is still open). Then you can import the unit2 module which is in the tutorial/python directory of the ACT-R software. If this is the same session you used before there will not be a notice that it has connected to ACT-R because it is already connected:

```
>>> import unit2
>>>
```

If this is a new session then it will print the confirmation that it has connected to ACT-R:

```
>>> import unit2
ACT-R connection has been started.
```

The unit2 module will also have ACT-R load the unit2-assignment-model.lisp file from the tutorial/unit2 directory, and if you look at the top of the Control Panel after you import the unit2 module you will see that it now says UNIT2 under “Current Model”.

Alternatively, if you have imported the actr module into Python you can use the "Import Python module" button in the Environment to pick a module to be imported into Python, and for this task the file is unit2.py in the python directory of the ACT-R tutorial.

To run the experiment you will call the **experiment** function from the unit2 module, and it has one optional parameter which indicates whether a human participant will be performing the task. As a first run you should perform the task yourself, and to do that you will call the **experiment** function at the Python prompt and pass it the value True:

```
>>> unit2.experiment(True)
```

When you enter that a window titled "Letter difference" will appear with three letters in it. When you press a key while that experiment window is the active window the experiment will record your key press and determine whether it was the correct response or not. If it was correct it will return the value True:

```
>>> unit2.experiment(True)
True
```

If it was not correct it will return False:

```
>>> unit2.experiment(True)
False
```

To run the model through the task you call the experiment function without providing a parameter. With the initial model that will result in this which returns False indicating an incorrect response (because the model did not make a response):

```
>>> unit2.experiment()
     0.000   GOAL                 SET-BUFFER-CHUNK GOAL (ISA READ-LETTERS STEP START) NIL
     0.000   VISION               SET-BUFFER-CHUNK VISUAL-LOCATION VISUAL-LOCATION0 NIL
     0.000   VISION               visicon-update
     0.000   PROCEDURAL           CONFLICT-RESOLUTION
     0.500   PROCEDURAL           CONFLICT-RESOLUTION
     0.500   ------               Stopped because no events left to process
False
```

2.10.3 Modeling task

Your task is to write a model that always responds correctly when performing the task. In doing this you should use the demo2 model as a guide. It reflects the way to interact with the imaginal, vision, and motor modules and the productions it contains are similar to the productions you will need to write. You will also need to write additional productions to read the other letters and decide which key to press.

You are provided with a chunk-type you may use for specifying the goal chunk, and the starting model already creates one and places it into the **goal** buffer. This chunk-type is the same as the one used in the demo2 model and only contains a slot named step:

```
(chunk-type read-letters step)
```

The initial goal provided looks just like the one used in demo2**:**

```
(goal-focus (isa read-letters step start))
```

There is an additional chunk-type specified which has slots for holding the three letters which can be used for the chunk in the **imaginal** buffer:

```
(chunk-type array letter1 letter2 letter3)
```

You do not have to use these chunk-types to solve the problem. If you have a different representation you would like to use feel free to do so. There is no one "right" model for the task, but for this assignment your model should follow the recommendation that any control information it uses is in the **goal** buffer and the problem representation is kept in the **imaginal** buffer. Otherwise, as long as the model is always correct at the task that is sufficient for this assignment.

In later units we will be comparing the model's performance to data from real experiments. Then, how well the model fits the data can be used as a way to decide between different representations and models, but that is not the only way to decide. Cognitive plausibility is another important factor when modeling human performance – you want the model to do the task in a way that is comparable to how a person does the task. A model that fits the data perfectly using a method completely unlike a person is usually not a very useful model of the task.

References

Anderson, J. R., Matessa, M., & Lebiere, C. (1997). ACT-R: A theory of higher level cognition and its relation to visual attention. *Human Computer Interaction, 12(4)*, 439-462.

Byrne, M. D., (2001). ACT-R/PM and menu selection: Applying a cognitive architecture to HCI. *International Journal of Human-Computer Studies, 55*, 41-84.

Kieras, D. & Meyer, D.E. (1997). An overview of the EPIC architecture for cognition and performance with application to human-computer interaction. *Human-Computer Interaction, 12*, 391-438.